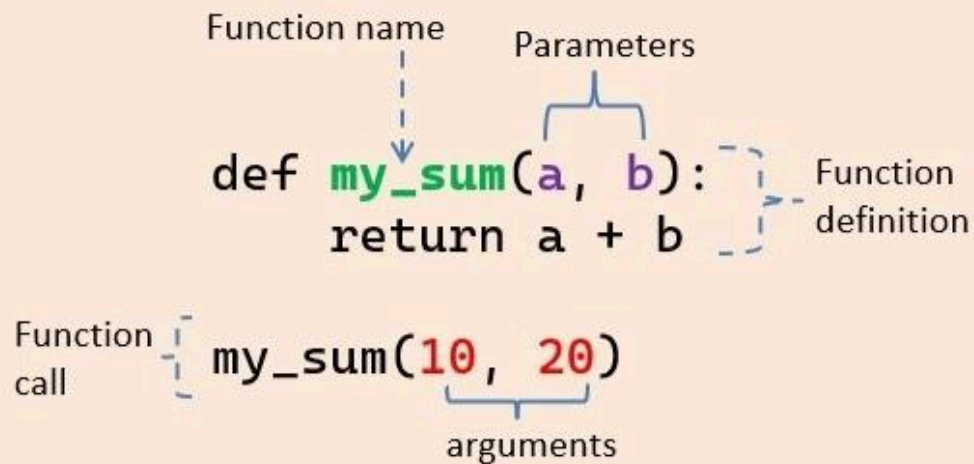


PART 3 — Functions

This is a **very important jump** in Python.

Part 1 taught you **objects/data**. Part 2 taught you **control flow**. Part 3 teaches you how to **package, reuse, pass around, and control logic**.



```
def my_sum(a, b):  
    return a + b
```

Function name

Parameters

Function definition

```
my_sum(10, 20)
```

Function call

arguments

Parameters are mentioned in the function definition.

Actual parameters(arguments) are passed during a function call

*args and **kwargs

Parameter 1

Parameter 2

Parameter 3

.....

***args** means passing an arbitrary number of arguments

****kwargs** means passing an arbitrary number of arguments with keywords



Parameter 1



Parameter 2



Parameter 3

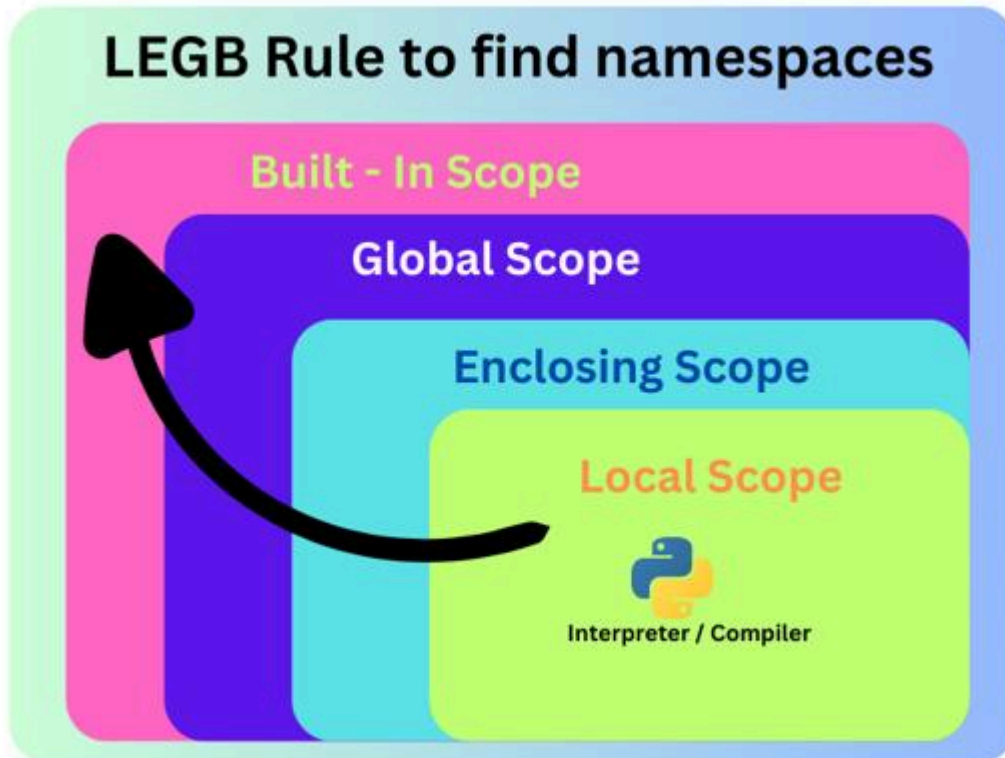


Parameter 4

.....



Preplnsta



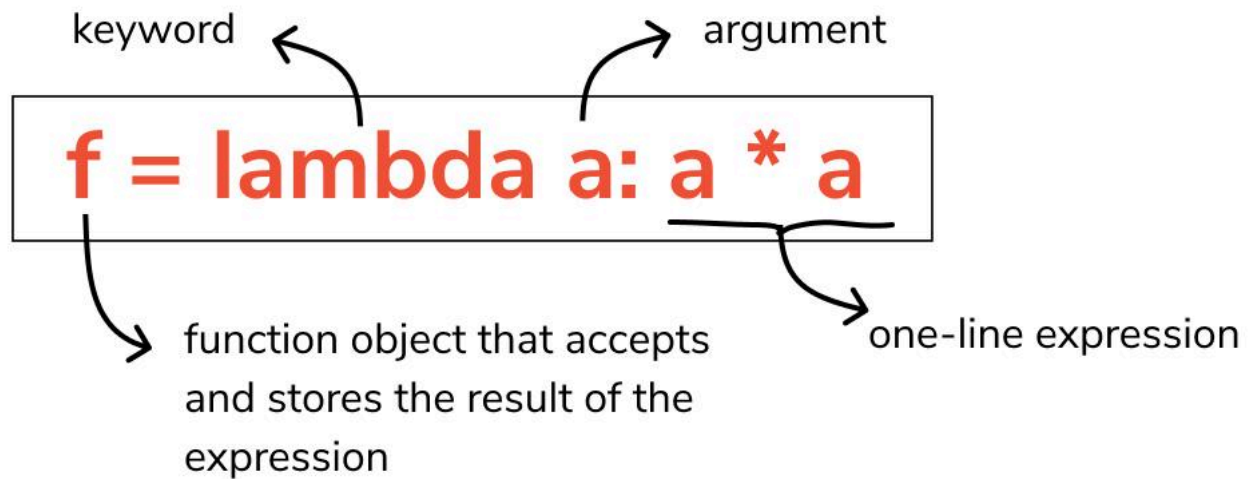
```
# higher order function that takes in another function  
def apply(func, ls):  
    return [func(i) for i in ls]
```

```
# using named function in higher order function  
def add10(x):  
    return x + 10  
print(apply(add10, [1, 2, 3]))
```

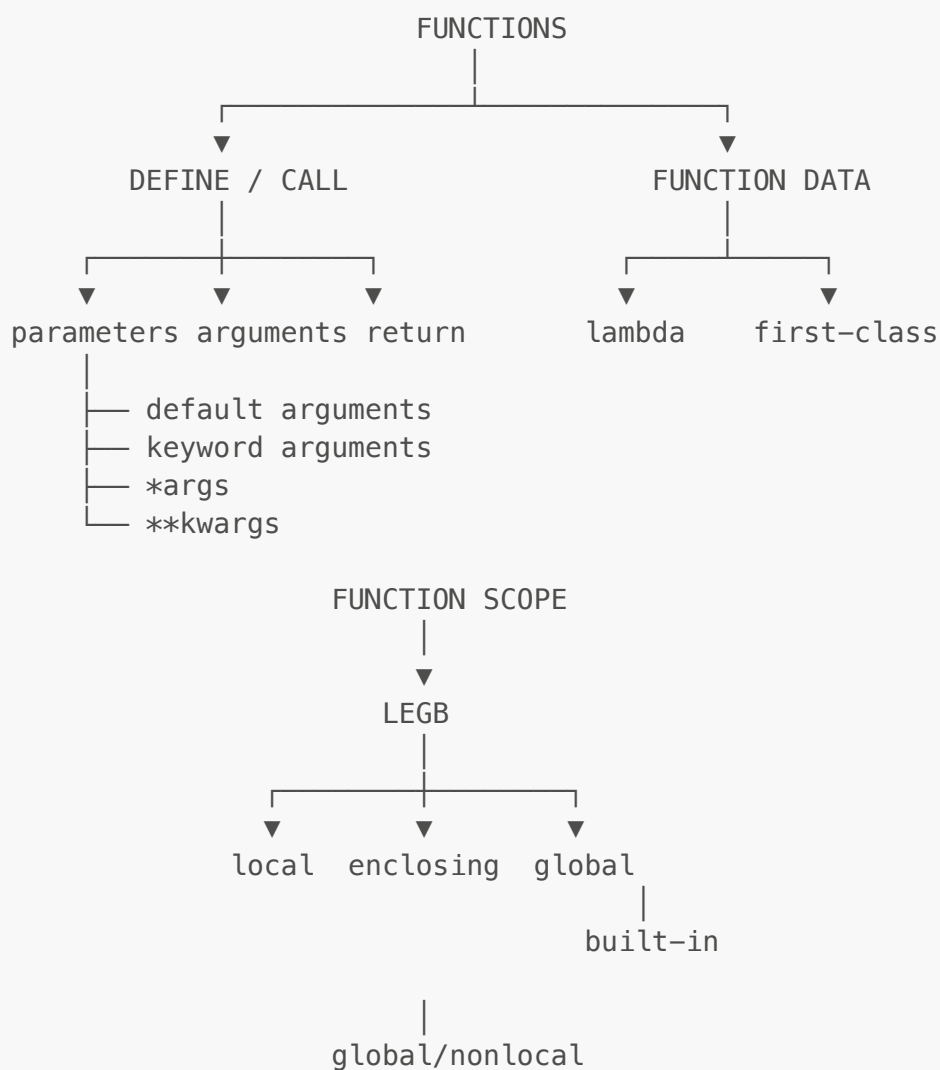
```
[11, 12, 13]
```

```
# using lambda function in higher order function  
print(apply(lambda x:x+10, [1, 2, 3]))
```

```
[11, 12, 13]
```



🧠 THE MASTER MAP



1. Functions

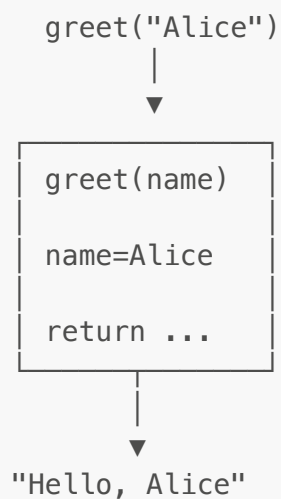
A function is a **reusable block of code**.

```
def greet(name):  
    return f"Hello, {name}"
```

Then call it:

```
message = greet("Alice")  
print(message)
```

Visual:



Function lifecycle

```
def function()  
    ↓  
CREATE function object  
    ↓  
call function()  
    ↓  
execute body  
    ↓  
return result
```

Important:

```
def hello():  
    print("Hello")
```

doesn't execute `hello()` yet.

It only defines the function.

Execution happens when:

```
hello()
```

2. Parameters vs Arguments

This distinction is extremely important.

```
def add(a, b):  
    return a + b
```

Here:

```
a, b → PARAMETERS
```

When you call:

```
add(10, 20)
```

then:

```
10, 20 → ARGUMENTS
```

Visual:

DEFINITION

```
def add(a, b):
```

↑ ↑
parameters

CALL

add(10, 20)

↑ ↑
arguments

Easy memory trick

Parameters = placeholders Arguments = actual values

3. *args

Suppose you don't know how many positional arguments you'll receive.

```
def total(*args):  
    print(args)
```

Call:

```
total(10, 20, 30)
```

Inside the function:

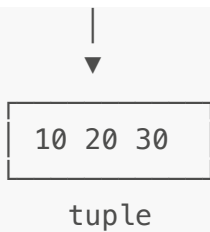
```
args
```

is a **tuple**:

```
(10, 20, 30)
```

Visual:

```
total(10, 20, 30)  
  |   |   |  
  └───┘  
  ▼  
 *args
```



Example:

```
def total(*args):  
    return sum(args)  
  
print(total(1, 2, 3))
```

Result:

6

★ Remember

*args → extra positional arguments → tuple

4. **kwargs

kwargs handles extra **keyword arguments**.

```
def show_info(**kwargs):  
    print(kwargs)
```

Call:

```
show_info(  
    name="Alice",  
    age=25,  
    city="Kolkata"  
)
```

Inside:


```
kwargs
```

becomes:

```
{  
    "name": "Alice",  
    "age": 25,  
    "city": "Kolkata"  
}
```

Visual:

```
show_info(  
    name="Alice",  
    age=25,  
    city="Kolkata"  
)
```

↓
▼
**kwargs
↓
▼

```
name → Alice  
age  → 25  
city → ...
```

dict

★ Remember

```
*args    → positional → tuple  
**kwargs → keyword    → dict
```

5. Default Arguments

You can provide a default value.

```
def greet(name, message="Hello"):
```

```
return f"{message}, {name}"
```

Now:

```
greet("Alice")
```

uses:

```
message = "Hello"
```

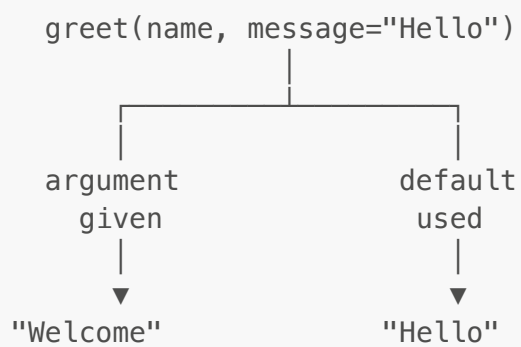
But:

```
greet("Alice", "Welcome")
```

uses:

```
message = "Welcome"
```

Visual:



Important rule

Non-default parameters must come before default parameters:

```
def greet(name, message="Hello"):  
    ...
```

Good.

But:

```
def greet(message="Hello", name):  
    ...
```

❌ Invalid.

6. Keyword Arguments

You can pass arguments by parameter name.

```
def user(name, age, city):  
    ...
```

Positional:

```
user("Alice", 25, "Kolkata")
```

Keyword:

```
user(  
    name="Alice",  
    age=25,  
    city="Kolkata"  
)
```

You can also change the order:

```
user(  
    city="Kolkata",  
    name="Alice",  
    age=25  
)
```

Visual:

POSITIONAL

```
user("Alice", 25, "Kolkata")
```



```
name    age    city
```

KEYWORD

```
user(  
    city="Kolkata",  
    name="Alice",  
    age=25  
)
```

↓ explicit mapping ↓

```
city → "Kolkata"  
name → "Alice"  
age  → 25
```

Mental model

```
positional → position matters  
keyword    → name matters
```

7. Scope

Scope answers:

Where can I access this variable?

Example:

```
x = 10  
  
def test():  
    y = 20  
    print(x)  
    print(y)
```

Visual:

GLOBAL SCOPE

```
x = 10
```

```
FUNCTION test()  
    |  
    |  
    |
```

```
y = 20
```

```
can access x
```

```
can access y
```

`y` is local to `test()`.

Outside:

```
print(y)
```

✗ `NameError`

8. LEGB

This is **critical**.

When Python sees:

```
print(x)
```

it searches for `x` using:

```
L → E → G → B
```

Meaning:

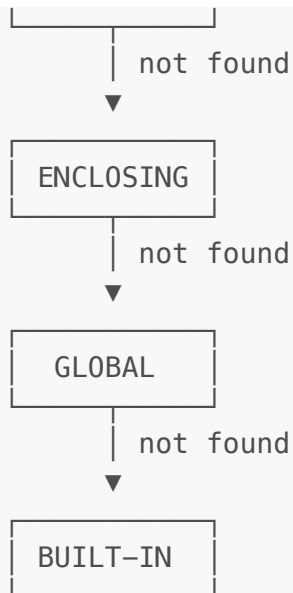
```
L = Local  
E = Enclosing  
G = Global  
B = Built-in
```

Visual:

x ?



LOCAL



Example:

```
x = "global"

def outer():
    x = "enclosing"

    def inner():
        x = "local"
        print(x)

    inner()

outer()
```

Output:

```
local
```

Because Python finds the local `x` first.

9. global / nonlocal

These become easy once LEGB is understood.

`global`

Means:

"I want to modify the variable from the global scope."

```
x = 10

def change():
    global x
    x = 20

change()

print(x)
```

Result:

20

Without `global`:

```
x = 10

def change():
    x = 20
```

Python treats `x` as a **local variable** inside the function.

nonlocal

Used with nested functions.

```
def outer():
    x = 10

    def inner():
        nonlocal x
        x += 1

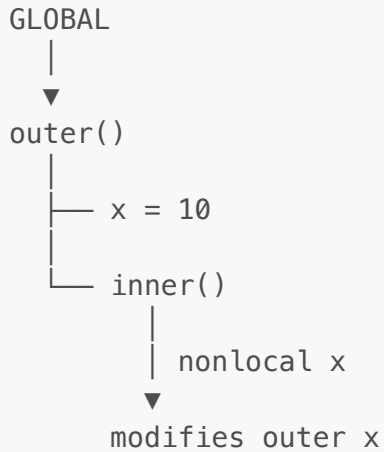
    inner()
    print(x)

outer()
```

Output:

11

Visual:



Remember

```

global    → module/global scope
nonlocal  → nearest enclosing function scope
  
```

10. Lambda

A lambda is a small anonymous function.

Normal function:

```

def square(x):
    return x * x
  
```

Lambda:

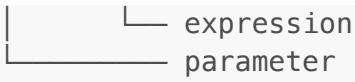
```

square = lambda x: x * x
  
```

Visual:

```

lambda x: x * x
  |      |
  
```

```
┌ expression
└ parameter
```

Example:

```
add = lambda a, b: a + b
print(add(3, 4))
```

Result:

7

Common usage:

```
numbers = [1, 2, 3, 4]
result = list(map(lambda x: x * 2, numbers))
```

Important

Lambda is intended for **small expressions**.

```
lambda x: x * 2
```

Good.

For complicated multi-step logic, use `def`.

11. First-Class Functions

This is a **big Python concept**.

In Python, functions are objects.

Therefore you can:

```
store them
pass them
```

```
return them
put them in collections
```

Example:

```
def greet(name):
    return f"Hello {name}"
```

Store function:

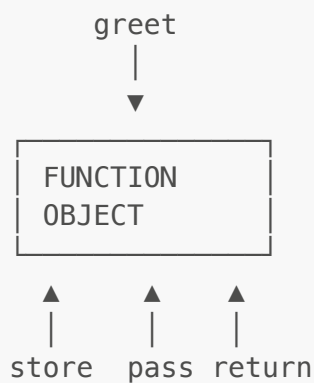
```
f = greet
```

Now:

```
f("Alice")
```

works.

Visual:



Function in list:

```
functions = [greet, print, len]
```

This works because functions are **first-class objects**.

12. Higher-Order Functions

A higher-order function is a function that:

takes another function as an argument OR returns a function.

Example 1:

```
def apply(func, value):  
    return func(value)
```

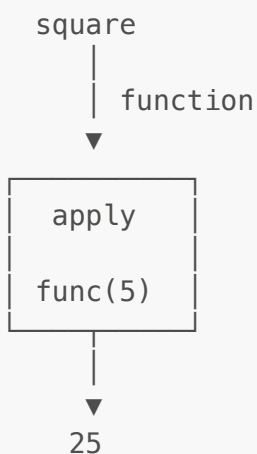
Then:

```
def square(x):  
    return x * x  
  
print(apply(square, 5))
```

Result:

25

Visual:



Higher-order function can return a function

```
def multiplier(factor):  
  
    def multiply(number):  
        return number * factor  
  
    return multiply
```

Then:

```
double = multiplier(2)

print(double(10))
```

Result:

20

This connects directly to **closures** from your earlier topic.

```
Higher-order function
  |
  ▼
returns inner function
  |
  ▼
Closure
  |
  ▼
remembers factor
```

*args + **kwargs together

You will see this constantly in professional Python.

```
def func(*args, **kwargs):
    print(args)
    print(kwargs)
```

Call:

```
func(1, 2, 3, name="Alice", age=25)
```

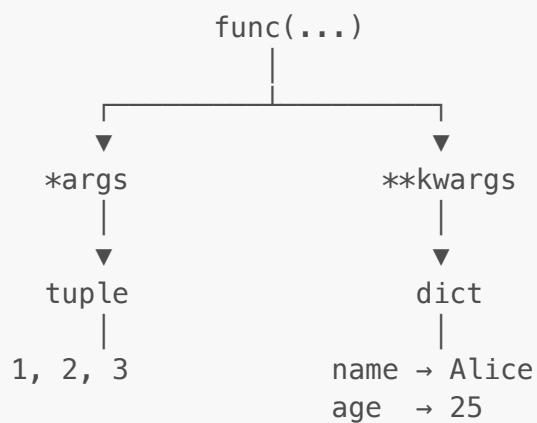
Result conceptually:

```
args
↓
```

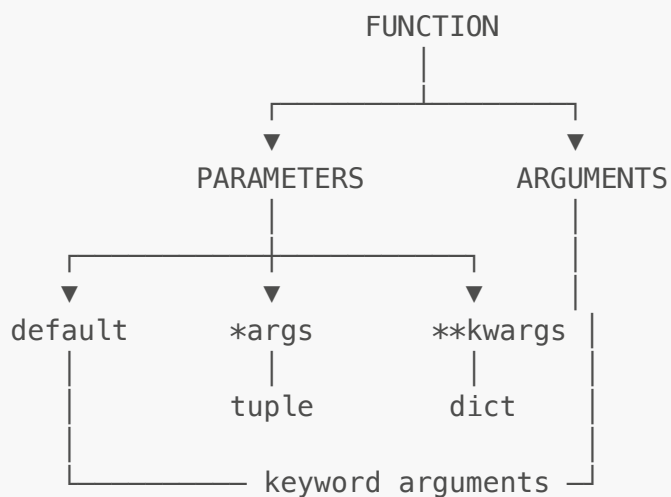
```
(1, 2, 3)

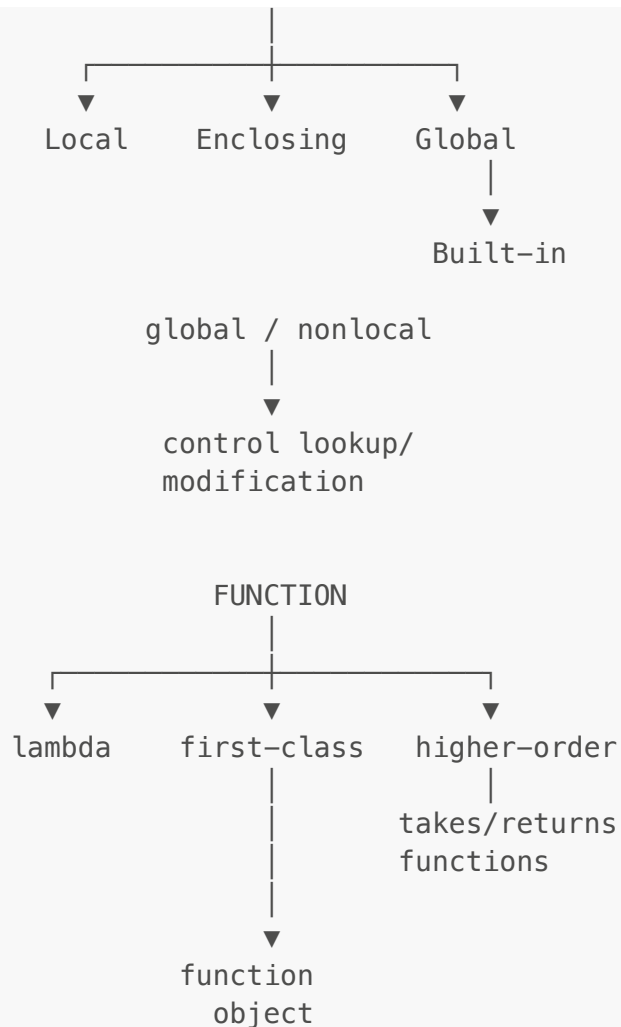
kwargs
↓
{
  "name": "Alice",
  "age": 25
}
```

Visual:



THE COMPLETE FUNCTION MAP





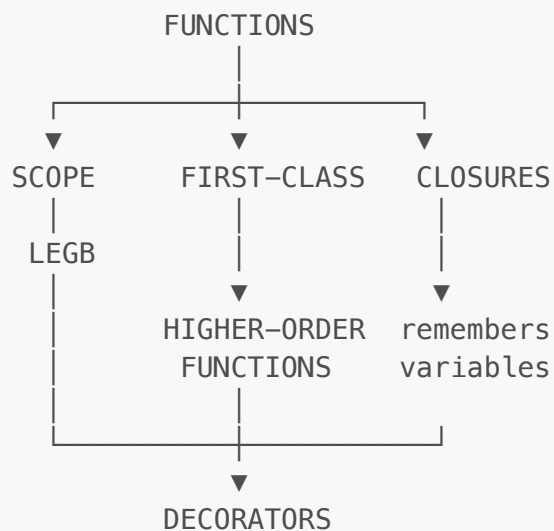
★ MASTER CHEAT SHEET

Topic	Remember
Function	Reusable block of code
Parameter	Placeholder in definition
Argument	Actual value passed
<code>*args</code>	Extra positional → tuple
<code>**kwargs</code>	Extra keyword → dict
Default argument	Used when argument isn't supplied
Keyword argument	Passed using parameter name
Scope	Where a name is accessible
LEGB	Local → Enclosing → Global → Built-in
<code>global</code>	Refer to/modify global variable
<code>nonlocal</code>	Refer to/modify enclosing variable

Topic	Remember
Lambda	Small anonymous function
First-class function	Function is an object
Higher-order function	Takes/returns a function

The 5 concepts that unlock the rest

If you really want to **master Python**, pay special attention to:



Once **scope** → **LEGB** → **closures** → **first-class functions** → **higher-order functions** clicks, decorators will stop feeling like Python magic. They become a logical combination of these concepts.